

The `solve()` function uses its `maxMoves` variable as the `movesRemaining` parameter in the initial call to `attemptMove()`. Each recursive call passes `maxMoves - 1` for the next value of `maxMoves`, causing it to decrease as more recursive calls are made. When it becomes less than 0, the `attemptMove()` function stops making additional recursive calls and returns `False`.

Finally, the `prevMove` parameter contains the `UP`, `DOWN`, `LEFT`, or `RIGHT` value that the previous call to `attemptMove()` made so that it doesn't undo that move. For the initial call to `attemptMove()`, the `solve()` function passes Python's `None` or JavaScript's `null` value for this parameter, since no previous move exists.

The beginning of the `attemptMove()` code checks for two base cases, returning `False` if `movesRemaining` has become less than 0, and returning `True` if `board` is in the solved state. The `SOLVED_BOARD` constant contains a board in the solved state that we can compare to the data structure in `board`.

The next part of `attemptMove()` performs each of the valid moves it can do on this board. The Python code is as follows:

Python

```
# RECURSIVE CASE - Attempt each of the valid moves:
for move in getValidMoves(board, prevMove):
    # Make the move:
    makeMove(board, move)
    movesMade.append(move)

    if attemptMove(board, movesMade, movesRemaining - 1, move):
        # If the puzzle is solved, return True:
        undoMove(board, move) # Reset to the original puzzle.
        return True
```

The JavaScript code is as follows:

JavaScript

```
// RECURSIVE CASE - Attempt each of the valid moves:
for (let move of getValidMoves(board, prevMove)) {
    // Make the move:
    makeMove(board, move);
    movesMade.push(move);

    if (attemptMove(board, movesMade, movesRemaining - 1, move)) {
```